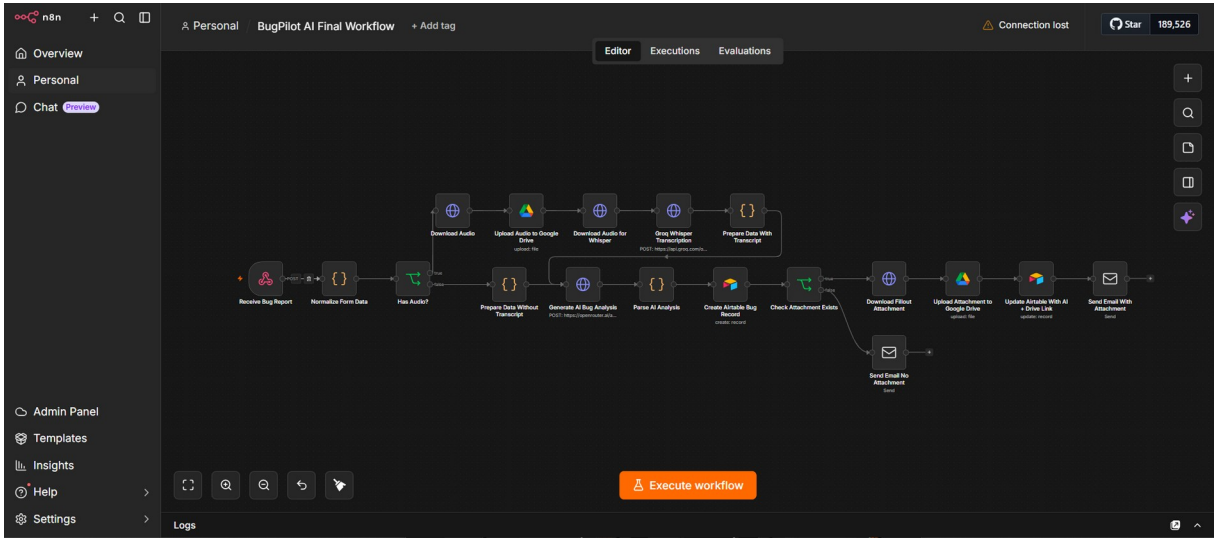


Bug Report → Çözüm Önerisi AI

Başak Balkan
Öğrenci No: 244410022
Bilgisayar Mühendisliği
Kastamonu Üniversitesi

Özet- Bu çalışma, kullanıcı tarafından bildirilen hata kayıtlarının tek bir iş akışı içerisinde toplanması, yapılandırılması, önceliklendirilmesi ve ilgili kanallara aktarılması amacıyla geliştirilen BugPilot AI sistemini ele almaktadır. Geliştirilen sistemde Fillout formu aracılığıyla alınan metin, ekran görüntüsü/dosya ve isteğe bağlı ses kaydı n8n üzerinde işlenmekte; ses verisi Groq Whisper servisiyle metne dönüştürülmekte, hata içeriği OpenRouter üzerinden yapay zekâ destekli olarak analiz edilmekte, sonuçlar Airtable üzerinde kayıt altına alınmakta, ek dosyalar Google Drive'a aktarılmakta ve süreç sonunda Gmail/SMTP üzerinden bildirim gönderilmektedir. Uygulama, sesli-sessiz ve ekli-eksiz kullanım senaryoları üzerinden doğrulanmış; temel kayıt, transkripsiyon, sınıflandırma, dosya saklama ve bildirim adımlarının tutarlı biçimde çalıştığı gözlemlenmiştir. Elde edilen bulgular, düşük kodlu araçlarla oluşturulan bütünselik otomasyon mimarilerinin küçük ölçekli yazılım ekiplerinde hata raporlama ve ilk triyaj sürecini destekleyebileceğini göstermektedir.

Anahtar Kelimeler- hata raporlama, bug triage, iş akışı otomasyonu, n8n, OpenRouter, Groq Whisper, Airtable, Google Drive, Gmail



BugPilot AI için geliştirilen n8n iş akışının genel görünümü.

I. GİRİŞ

Yazılım geliştirme süreçlerinde hata bildirimleri çoğunlukla farklı kaynaklardan ve farklı biçimlerde alınmaktadır. Kullanıcı e-postaları, mesajlaşma uygulamaları, ekran görüntüleri, kısa açıklamalar ve ses kayıtları bu bildirimlere örnek olarak verilebilir. Bu verilerin manuel olarak incelenmesi, sınıflandırılması, önceliklendirilmesi ve geliştiriciye aktarılması hem zaman kaybına hem de süreç tutarsızlıklarına neden olabilmektedir. Bu nedenle hata raporlarının standart bir yapıya dönüştürülmesi ve ilk değerlendirme aşamasının otomatikleştirilmesi yazılım bakım süreçleri açısından önem taşımaktadır.

Bu çalışma kapsamında geliştirilen BugPilot AI sistemi, hata raporlarını merkezi bir form üzerinden almakta ve rapor içeriğini otomatik bir iş akışıyla işlemektedir. Sistem; hata başlığı

B. Ses Transkripsiyonu

Sesli açıklama alanı isteğe bağlıdır. Has Audio? IF node, audio_url alanının dolu olup olmadığını değerlendirir. Ses dosyası varsa içerik HTTP Request ile binary biçimde indirilir, Google Drive klasörüne yüklenir ve Groq Whisper Transcription node tarafından metne dönüştürülür. Prepare Data With Transcript node, elde edilen metni transcript alanına ekleyerek yapay zekâ analizine hazır hale getirir.

II. ARKA PLAN VE KULLANILAN ARAÇLAR

BugPilot AI, düşük kodlu otomasyon araçları ile API tabanlı servislerin bir araya getirildiği bütünleşik bir mimariye dayanmaktadır. n8n Webhook node, harici sistemlerden gelen olay verileriyle iş akışını başlatmakta ve test/production URL ayrımını desteklemektedir [1]. Fillout, form gönderimlerinin webhook aracılığıyla üçüncü taraf sistemlere aktarılmasını sağladığı için veri girişi katmanı olarak kullanılmıştır [2]. Ses verilerinin metne dönüştürülmesi için Groq'un speech-to-text servisi tercih edilmiştir [3]. Hata raporunun özetlenmesi ve yapılandırılmış çıktı üretilmesi aşamasında OpenRouter chat completions arayüzünden yararlanılmıştır [4]. Hata kayıtlarının tablo yapısında saklanması için Airtable personal access token altyapısı kullanılmıştır [5]. Dosyaların saklanması ve paylaşılabilir bağlantı üretimi için Google Drive API, bildirim gönderimi için ise Gmail/SMTP yapılandırması kullanılmıştır [6], [7].

Söz konusu bileşenlerin birlikte kullanılmasıyla manuel bug raporlama süreci, ilk değerlendirmesi otomatik olarak yapılan bir triyaj hattına dönüştürülmüştür. Literatürdeki bug triage ve hata çözüm süresi tahmini çalışmaları, hata kayıtlarının düzenli, yapılandırılmış ve izlenebilir tutulmasının sonraki karar süreçlerini desteklediğini göstermektedir [8], [9]. Bu bağlamda geliştirilen sistem, veri toplama, analiz, kayıt, dosya yönetimi ve bildirim adımlarını tek bir operasyonel akışta birleştirmektedir.

III. SİSTEM MİMARİSİ

Sistemin genel mimarisi olay tabanlı bir iş akışı olarak tasarlanmıştır. Fillout formu gönderildiğinde n8n üzerinde tanımlanan production webhook tetiklenir. İlk aşamada Normalize Form Data node'u, Fillout tarafından gönderilen soru-cevap yapısını name, email, title, description, attachment_url, audio_url ve transcript gibi sabit alanlara dönüştürür. Bu normalizasyon katmanı, sonraki adımların form servisinin ham veri yapısına doğrudan bağlı kalmasını engeller.

Ses dosyası bulunması halinde akış true koluna yönlendirilir: ses dosyası indirilir, Google Drive klasörüne kaydedilir, Groq Whisper servisine gönderilir ve elde edilen transcript alanı AI analizine dahil edilir. Ses kaydı bulunmadığında transcript alanı boş bırakılır ve süreç kesintisiz olarak devam eder. Benzer şekilde, ekran görüntüsü veya ek dosya bulunması durumunda Google Drive yükleme adımı çalışır; dosya bulunmadığında ise sistem dosyasız bildirim koluna geçerek hata üretmeden tamamlanır. Şekil 1, bu dallanma yapısının n8n arayüzündeki son halini göstermektedir.

IV. UYGULAMA

A. Form Alımı ve Normalizasyon

Fillout formu ; kullanıcı bilgileri hata ayrıntıları, cihaz/tarayıcı bilgisi, ekran görüntüsü/dosya ve sesli açıklama alanlarından oluşmaktadır. n8n içerisindeki Normalize Form Data code node, Fillout tarafından gönderilen iç içe geçmiş soru-cevap yapısını standart JSON alanlarına dönüştürmektedir. Bu katman sayesinde sonraki node adımları, form alanlarının ham yapısından bağımsız ve daha kararlı biçimde çalışmaktadır. açıklama, tekrar etme adımları, beklenen ve gerçekleşen sonuç, cihaz/tarayıcı bilgisi, isteğe bağlı dosya ve isteğe bağlı sesli açıklama alanlarını kullanmaktadır. n8n üzerinde oluşturulan akış, bu verileri standart JSON alanlarına dönüştürmekte; ses verisi bulunması durumunda transkripsiyon üretmekte, hata içeriğini yapay zekâ modeliyle yorumlamakta ve sonuçları veri tabanı, dosya depolama ve e-posta bildirim katmanlarına aktarmaktadır. Çalışmanın temel amacı, yalnızca servisler arasında veri aktarımı yapmak değil; eksik ek, ses kaydı bulunmaması veya farklı girdi kombinasyonları gibi gerçek kullanım koşullarında da kesintisiz çalışabilen, izlenebilir ve genişletilebilir bir hata raporlama altyapısı ortaya koymaktır.

C. Yapay Zeka Destekli Hata Analizi

Generate AI Bug Analysis node, OpenRouter chat completions endpointine yapılandırılmış JSON isteği göndermektedir. Prompt içerisinde raporlayan kişi, ürün/sayfa, hata başlığı, açıklama, tekrar adımları, beklenen/gerçekleşen sonuç, cihaz/tarayıcı bilgisi ve varsa ses transkripti yer almaktadır. Modelden serbest metin yerine doğrudan geçerli JSON döndürmesi istenmiştir. Parse AI Analysis node ise bu cevabı temizleyerek summary, category, severity, possible_cause, suggested_solution, developer_notes, missing_information ve user_reply alanlarına ayırmaktadır. Böylece Airtable ve e-posta adımlarında standart bir veri yapısı kullanılmaktadır.

D. Veri Depolama ve Bildirim

Create Airtable Bug Record node; temel hata bilgilerini, transcript alanını ve yapay zekâ analiz çıktılarının metin alanlarını Airtable Bug Reports tablosuna yazmaktadır. Check Attachment Exists node dosya alanını kontrol eder. Dosya mevcutsa ilgili içerik indirilir, Google Drive içerisindeki BugPilot AI Attachments klasörüne yüklenir ve Airtable kaydındaki Attachment URL alanına Drive bağlantısı eklenir. Dosya mevcut değilse Attachment URL alanı boş kalır. Her iki durumda da sonuç bildirimini Gmail/SMTP üzerinden gönderilmektedir.

V. TEST SENARYOLARI VE SONUÇLAR

Sistem, üç temel kullanım senaryosu üzerinden manuel olarak test edilmiştir: (i) dosyalı ve sesli hata raporu, (ii) dosyasız fakat sesli hata raporu ve (iii) dosyasız-sessiz hata raporu. Her senaryoda n8n execution sonucu, Airtable kaydının oluşması, Google Drive dosya çıktıları, ses transkriptinin üretilmesi ve e-posta bildiriminin gönderilmesi ayrı ayrı kontrol edilmiştir.

Test sonuçları, sistemin tek bir başarılı yola bağlı kalmadığını göstermektedir. Dosya veya ses alanlarının boş bırakıldığı durumlarda ilgili IF kolları devreye girmiş ve iş akışı kesintiye uğramadan tamamlanmıştır. Bu durum, son kullanıcıların rapor gönderirken her alanı eksiksiz doldurmadığı gerçek kullanım koşulları için önemlidir. Ayrıca sesli senaryolarda üretilen transkriptin, hata açıklamasını zenginleştirerek analiz adımı ek bağlam sağladığı görülmüştür.

VI. TARTIŞMA

BugPilot AI, tamamen otomatik hata keşfi yapan bir monitoring sistemi olarak değil, manuel hata bildirimi girişini otomatik triyaj ve dağıtım sürecine dönüştüren bir sistem olarak değerlendirilmelidir. Sistem, kullanıcının raporladığı problemi yapılandırır, geliştirici için daha okunabilir ve izlenebilir hale getirir, ilgili ekleri merkezi depolara aktarır ve bildirim üretir. Literatürde yer alan bug triage çalışmaları da, hata kayıtlarının doğru yapılandırılmasının karar verme ve sorumluluk atama süreçlerini desteklediğini göstermektedir [8], [9].

Mimari açıdan önemli bir diğer nokta, API anahtarlarının açık metin çıktılara veya form verisine gömülmemesi ve n8n credential alanlarında saklanmasıdır. Bunun yanında production webhook URL kullanımı, test URL bağımlılığını ortadan kaldırarak sistemin gerçek form gönderimlerinde otomatik çalışmasını sağlamaktadır. Bu tercih, hem süreklilik hem de güvenlik bakımından kritik bir uygulama kararıdır.

VII. SINIRLILIKLAR VE GELECEK ÇALIŞMALAR

Mevcut sistemin çıktıları, hata raporunda verilen bilginin kapsamına ve açıklık düzeyine bağlıdır. Eksik veya belirsiz kullanıcı açıklamaları sistem tarafından missing_information alanında işaretlenebilmekte; ancak üretilen analiz, hatanın kod seviyesindeki kesin nedenini tek başına kanıtlamamaktadır. Gelecekte uygulama logları, ekran kaydı analizi, otomatik issue oluşturma, Slack/Discord bildirimi ve destek görüşmesi yönlendirmesi gibi eklentilerle sistem genişletilebilir.

Gelecek çalışmalarda sistem çıktılarının nicel kalite ölçütleriyle değerlendirilmesi de önem taşımaktadır. Örneğin severity tahminlerinin insan değerlendirmeleriyle karşılaştırılması, kategori tutarlılığının ölçülmesi ve yanlış pozitif/negatif analizlerin izlenmesi sistem güvenilirliğini artırabilir. Ayrıca uzun süreli kullanım verisi biriktikçe, bug triage ve çözüm süresi tahmini gibi ek karar destek modülleri geliştirilebilir [8], [9].

VIII. SONUÇ

Bu çalışmada, Fillout, n8n, OpenRouter, Groq Whisper, Airtable, Google Drive ve Gmail/SMTP kullanılarak hata raporlarını toplama, yapılandırma, analiz etme ve ilgili kişilere ulaşırma görevlerini tek bir iş akışında birleştiren BugPilot AI sistemi tasarlanmış ve uygulanmıştır. Geliştirilen çözüm; form verilerini normalize etme, ses transkripti üretme, yapılandırılmış analiz çıktısı elde etme, Airtable kaydı oluşturma, dosyaları Google Drive'da saklama ve e-posta bildirimi gönderme adımlarını uçtan uca gerçekleştirmektedir. Ekli-eksiz ve sessiz test senaryolarında elde edilen başarılı sonuçlar, sistemin gerçek kullanıcı davranışlarına karşı dayanıklı ve geliştirilebilir bir altyapı sunduğunu göstermektedir.

KAYNAKÇA

- [1] n8n Docs, "Webhook node," n8n Documentation. [Online]. Available: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>
- [2] Fillout Help, "Send form submissions to a webhook," Fillout Documentation. [Online]. Available: <https://fillout.com/help/webhook>
- [3] Groq, "Speech to Text," GroqDocs. [Online]. Available: <https://console.groq.com/docs/speech-to-text>
- [4] OpenRouter, "Create a chat completion," OpenRouter Documentation. [Online]. Available: <https://openrouter.ai/docs/api/api-reference/chat/send-chat-completion-request>
- [5] Airtable Support, "Creating personal access tokens," Airtable Documentation. [Online]. Available: <https://support.airtable.com/docs/creating-personal-access-tokens>
- [6] Google, "Google Drive API overview," Google for Developers. [Online]. Available: <https://developers.google.com/workspace/drive/api/guides/about-sdk>
- [7] Google, "Add Gmail to another email client," Gmail Help. [Online]. Available: <https://support.google.com/mail/answer/7126229>
- [8] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in Proceedings of the 28th International Conference on Software Engineering (ICSE'06), Shanghai, China, 2006, pp. 361-370, doi: 10.1145/1134285.1134336.
- [9] E. Giger, M. D'Ambrosio, M. Pinzger, and H. C. Gall, "Predicting the Fix Time of Bugs," in Proceedings of the 2nd International Workshop on Recommendation Systems for Software Engineering, Cape Town, South Africa, 2010, pp. 52-56, doi: 10.1145/1808920.1808933.
- [10] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), Atlanta, GA, USA, 2008, pp. 308-318, doi: 10.1145/1453101.1453146.
- [11] S. Breu, R. Premraj, J. Sillito, and T. Zimmermann, "Information needs in bug reports: improving cooperation between developers and users," in Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW), Savannah, GA, USA, 2010, pp. 301-310, doi: 10.1145/1718918.1718973.
- [12] P. Runeson, M. Alexandersson, and O. Nyholm, "Detection of duplicate defect reports using natural language processing," in Proceedings of the 29th International Conference on Software Engineering (ICSE), Minneapolis, MN, USA, 2007, pp. 499-510, doi: 10.1109/ICSE.2007.32.
- [13] N. Jalbert and W. Weimer, "Automated duplicate detection for bug tracking systems," in Proceedings of the IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN), Anchorage, AK, USA, 2008, pp. 52-61, doi: 10.1109/DSN.2008.4630070.
- [14] C. Sun, D. Lo, S.-C. Khoo, and J. Jiang, "Towards more accurate retrieval of duplicate bug reports," in Proceedings of the 26th IEEE/ACM International Conference on Automated Software Engineering (ASE), Lawrence, KS, USA, 2011, pp. 253-262.
- [15] J. Zhou, H. Zhang, and D. Lo, "Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports," in Proceedings of the 34th International Conference on Software Engineering (ICSE), Zurich, Switzerland, 2012, pp. 14-24, doi: 10.1109/ICSE.2012.6227210.
- [16] R. K. Saha, M. Lease, S. Khurshid, and D. E. Perry, "Improving bug localization using structured information retrieval," in Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering (ASE), Palo Alto, CA, USA, 2013, pp. 345-355.